

実験 第2回レポート

学籍番号 2022311042 名前 神野友哉
2023年10月18日

1 実験 2-1

実験 2-1 に用いた平滑化フィルタを説明し、出力画像を示せ。

1.1 考察



図 1: image2.png



図 2: 平滑化後

ソースコード 1: heikatu-pgm.c

```

1 // pgm ファイルから画像データの読込とファイルへの書き出し
2
3 #include <stdio.h>
4 #include <string.h>
5 #include <stdlib.h>
6
7 int main(void) {
8     FILE *fpi = NULL; // 入力画像
9     FILE *fpo = NULL; // 出力画像
10
11     char buf[258];
12     int width, height; // 画像の横長と縦長
13     int maxvalue; // 画像の最大輝度値
14     int i, j, x, y;
15     int count;
16     char fnamein[50];
17     char fnamein_buf[50];
18     char fnameout[50];
19     char fnameout_right[10];
20     char *tok;
21
22     puts("what name is your file? Example name.pgm");
23     scanf("%s", fnamein);
24     for (i = 0; i < sizeof(fnamein_buf); i++) fnamein_buf[i] = fnamein[i];
25
26     tok = strtok(fnamein_buf, ".");
27     sprintf(fnameout, "%s-heikatu-out.", tok);
28     tok = strtok(NULL, ".");
29     sprintf(fnameout_right, "%s", tok);
30     strcat(fnameout, fnameout_right);
31
32     // ファイル fnamein から画像データを読み込む

```

```

33  fpi = fopen(fnamein, "r");
34
35  if (fpi == NULL) printf("Reading file open error ! %s\n", fnamein);
36
37  do {
38      fgets(buf, 256, fpi);
39  } while (buf[0] == '#'); // skip over comments
40
41  if (buf[0] != 'P' || buf[1] != '2') {
42      fclose(fpi);
43      printf(" \n The input image is not pgm format! \n\n");
44      return -1;
45  }
46
47  do {
48      fgets(buf, 256, fpi);
49  } while (buf[0] == '#'); // skip over comments
50
51  sscanf(buf, "%d %d", &width, &height); // 画像サイズを読み込む
52  fscanf(fpi, "%d", &maxvalue); // 最大の輝度値を読み込み
53
54  int **img;
55  int **bufArr;
56  img = (int **)calloc(height, sizeof(int *));
57  bufArr = (int **)calloc(height, sizeof(int *));
58  for (i = 0; i < height; i++) {
59      img[i] = (int *)calloc(width, sizeof(int));
60      bufArr[i] = (int *)calloc(width, sizeof(int));
61  }
62  if (img == NULL || bufArr == NULL) {
63      printf("failed to get enough of memory\n");
64      return -1;
65  }
66
67  for (y = 0; y < height; y++) {
68      for (x = 0; x < width; x++) {
69          fscanf(fpi, "%d", &img[y][x]);
70          bufArr[y][x] = 0;
71      }
72  } // 画像中 (x,y)の位置にある画素をimg[y][x]で参照できる。
73
74  fclose(fpi);
75
76  int x_l, y_u;
77  int x_r, y_d;
78  int value_sum;
79  for (y = 0; y < height; y++) {
80      for (x = 0; x < width; x++) {
81          value_sum = 0;
82          count = 0;

```

```

83     x_l = y_u = -1;
84     x_r = y_d = 1;
85     if (x == 0) x_l = 0;
86     if (x == width - 1) x_r = 0;
87     if (y == 0) y_u = 0;
88     if (y == height - 1) y_d = 0;
89
90     for (j = y_u; j <= y_d; j++) {
91         for (i = x_l; i <= x_r; i++) {
92             value_sum += img[y + j][x + i];
93             count++;
94         }
95     }
96     bufArr[y][x] = value_sum / count;
97 }
98 }
99
100 for (y = 0; y < height; y++) {
101     for (x = 0; x < width; x++) {
102         img[y][x] = bufArr[y][x];
103     }
104 }
105
106 // 画像データに対する処理, 各要素の値を+ 30
107 // ただし、画素値が maxvalue より大きい場合、maxvalue にする
108 /*
109 for(y=0; y<height; y++) {
110     for(x=0; x<width; x++) {
111         img[y][x] = img[y][x]+30;
112         if(img[y][x] > maxvalue)
113             img[y][x] = maxvalue;
114     }
115 }
116 */
117 // 画像データをファイル fnameout に出力する
118
119 fpo = fopen(fnameout, "w");
120
121 if (fpo == NULL) printf("Reading file open error ! %s\n", fnameout);
122
123 fprintf(fpo, "P2\n");
124 fprintf(fpo, "# %s\n", fnameout);
125 fprintf(fpo, "%d %d", width, height);
126 fprintf(fpo, " \n");
127 fprintf(fpo, "%d", maxvalue);
128 fprintf(fpo, " \n");
129
130 count = 0;
131
132 for (y = 0; y < height; y++) {

```

```
133     for (x = 0; x < width; x++) {
134         fprintf(fpo, "%3d ", img[y][x]);
135         count++;
136         if (count % 18 == 0) fprintf(fpo, "\n"); // 1行 18個画素
137     }
138 }
139 fclose(fpo); // 出力画像をクローズする
140
141 for(i = 0; i < height; i++) {
142     free(img[i]);
143     free(bufArr[i]);
144 }
145 free(img);
146 free(bufArr);
147 }
```

2 実験 2-2

実験 2-2 に用いた先鋭化フィルタを説明し、出力画像を示せ。

2.1 考察



図 3: image2.png



図 4: 先鋭化後

ソースコード 2: senei-pgm.c

```

1 // pgm ファイルから画像データの読込とファイルへの書き出し
2
3 #include <stdio.h>
4 #include <string.h>
5 #include <stdlib.h>
6
7 int main(void) {
8     FILE *fpi = NULL; // 入力画像
9     FILE *fpo = NULL; // 出力画像
10
11     char buf[258];
12     int width, height; // 画像の横長と縦長
13     int maxvalue; // 画像の最大輝度値
14     int i, j, x, y;
15     int count;
16     char fnamein[50];
17     char fnamein_buf[50];
18     char fnameout[50];
19     char fnameout_right[10];
20     char *tok;
21
22     puts("what name is your file? Example name.pgm");
23     scanf("%s", fnamein);
24     for (i = 0; i < sizeof(fnamein_buf); i++) {
25         fnamein_buf[i] = fnamein[i];
26     }
27     tok = strtok(fnamein_buf, ".");
28     sprintf(fnameout, "%s-senei-out.", tok);
29     tok = strtok(NULL, ".");
30     sprintf(fnameout_right, "%s", tok);
31     strcat(fnameout, fnameout_right);
32

```

```

33 // ファイル fnamein から画像データを読み込む
34 fpi = fopen(fnamein, "r");
35
36 if (fpi == NULL)
37     printf("Reading file open error ! %s\n", fnamein);
38
39 do {
40     fgets(buf, 256, fpi);
41 } while (buf[0] == '#'); // skip over comments
42
43 if (buf[0] != 'P' || buf[1] != '2') {
44     fclose(fpi);
45     printf(" \n The input image is not pgm format! \n\n");
46     return -1;
47 }
48
49 do {
50     fgets(buf, 256, fpi);
51 } while (buf[0] == '#'); // skip over comments
52
53 sscanf(buf, "%d %d", &width, &height); // 画像サイズを読み込む
54 fscanf(fpi, "%d", &maxvalue); // 最大の輝度値を読み込み
55
56 int **img;
57 int **bufArr;
58 img = (int **)calloc(height, sizeof(int *));
59 bufArr = (int **)calloc(height, sizeof(int *));
60 for (i = 0; i < height; i++) {
61     img[i] = (int *)calloc(width, sizeof(int));
62     bufArr[i] = (int *)calloc(width, sizeof(int));
63 }
64 if (img == NULL || bufArr == NULL) {
65     printf("failed to get enough of memory\n");
66     return -1;
67 }
68
69 for (y = 0; y < height; y++) {
70     for (x = 0; x < width; x++) {
71         fscanf(fpi, "%d", &img[y][x]);
72         bufArr[y][x] = 0;
73     }
74 } // 画像中 (x,y)の位置にある画素をimg[y][x]で参照できる。
75
76 fclose(fpi);
77
78 int x_l, y_u;
79 int x_r, y_d;
80 int value_sum, sharpen_weight;
81 int weight;
82

```



```

83     do {
84         puts("write sharpen-weight under 0. Example -1");
85         scanf("%d", &sharpen_weight);
86     } while (sharpen_weight >= 0);
87
88     for (y = 0; y < height; y++) {
89         for (x = 0; x < width; x++) {
90             value_sum = 0;
91             x_l = y_u = -1;
92             x_r = y_d = 1;
93             count = 4;
94             if (x == 0) {
95                 x_l = 0;
96                 count--;
97             }
98             if (x == width - 1) {
99                 x_r = 0;
100                count--;
101            }
102            if (y == 0) {
103                y_u = 0;
104                count--;
105            }
106            if (y == height - 1) {
107                y_d = 0;
108                count--;
109            }
110
111            for (j = y_u; j <= y_d; j++) {
112                for (i = x_l; i <= x_r; i++) {
113                    weight = 0;
114                    if (!j || !i) weight = sharpen_weight;
115                    if (!j && !i) weight = 1 - count * sharpen_weight;
116                    value_sum += (weight * img[y + j][x + i]);
117                }
118            }
119            value_sum = (value_sum > maxvalue) ? maxvalue : value_sum;
120            bufArr[y][x] = (value_sum < 0) ? 0 : value_sum;
121        }
122    }
123
124    for (y = 0; y < height; y++) {
125        for (x = 0; x < width; x++) {
126            img[y][x] = bufArr[y][x];
127        }
128    }
129
130    // 画像データに対する処理, 各要素の値を+ 30
131    // ただし、画素値が maxvalue より大きい場合、maxvalue にする
132    /*

```

```

133     for(y=0; y<height; y++) {
134         for(x=0; x<width; x++) {
135             img[y][x] = img[y][x]+30;
136             if(img[y][x] > maxvalue)
137                 img[y][x] = maxvalue;
138         }
139     }
140     */
141     // 画像データをファイル fnameout に出力する
142
143     fpo = fopen(fnameout, "w");
144
145     if (fpo == NULL) printf("Reading file open error ! %s\n", fnameout);
146
147     fprintf(fpo, "P2\n");
148     fprintf(fpo, "# %s\n", fnameout);
149     fprintf(fpo, "%d %d", width, height);
150     fprintf(fpo, " \n");
151     fprintf(fpo, "%d", maxvalue);
152     fprintf(fpo, " \n");
153
154     count = 0;
155
156     for (y = 0; y < height; y++) {
157         for (x = 0; x < width; x++) {
158             fprintf(fpo, "%3d ", img[y][x]);
159             count++;
160             if (count % 18 == 0) {
161                 fprintf(fpo, "\n"); // 1行 18個画素
162             }
163         }
164     }
165     fclose(fpo); // 出力画像をクローズする
166
167     for(i = 0; i < height; i++) {
168         free(img[i]);
169         free(bufArr[i]);
170     }
171     free(img);
172     free(bufArr);
173 }

```

3 実験 2-3

実験 2-3 に用いたエッジ検出方法を説明し、入力画像は原画像、平滑化された画像と先鋭化された画像であるときの出力画像を示せ。

3.1 考察



図 5: image2.png



図 6: 入力 原画像



図 7: 入力 平滑

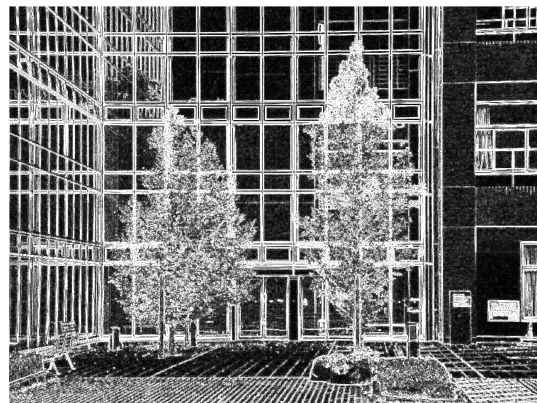


図 8: 入力 先鋭

ソースコード 3: sobel-pgm.c

```

1 // pgm ファイルから画像データの読込とファイルへの書き出し
2
3 #include <stdio.h>
4 #include <string.h>
5 #include <math.h>
6 #include <stdlib.h>
7
8 #define DEGREE_OF_EDGE 1
9
10 int main(void) {
11     FILE *fpi = NULL; // 入力画像
12     FILE *fpo = NULL; // 出力画像
13
14     char buf[258];
15     int width, height; // 画像の横長と縦長
16     int maxvalue; // 画像の最大輝度値

```

```

17     int i, j, x, y;
18     char fnamein[50];
19     char fnamein_buf[50];
20     char fnameout[50];
21     char fnameout_right[10];
22     char *tok;
23
24     puts("what name is your file? Example name.pgm");
25     scanf("%s", fnamein);
26     for (i = 0; i < sizeof(fnamein_buf); i++) {
27         fnamein_buf[i] = fnamein[i];
28     }
29     tok = strtok(fnamein_buf, ".");
30     sprintf(fnameout, "%s-sobel-out.", tok);
31     tok = strtok(NULL, ".");
32     sprintf(fnameout_right, "%s", tok);
33     strcat(fnameout, fnameout_right);
34
35     // ファイル fnamein から画像データを読み込む
36     fpi = fopen(fnamein, "r");
37
38     if (fpi == NULL) printf("Reading file open error ! %s\n", fnamein);
39
40     do {
41         fgets(buf, 256, fpi);
42     } while (buf[0] == '#'); // skip over comments
43
44     if (buf[0] != 'P' || buf[1] != '2') {
45         fclose(fpi);
46         printf(" \n The input image is not pgm format! \n\n");
47         return -1;
48     }
49
50     do {
51         fgets(buf, 256, fpi);
52     } while (buf[0] == '#'); // skip over comments
53
54     sscanf(buf, "%d %d", &width, &height); // 画像サイズを読み込む
55     fscanf(fpi, "%d", &maxvalue); // 最大の輝度値を読み込み
56
57     int **img;
58     int **bufArr;
59     img = (int **)calloc(height, sizeof(int *));
60     bufArr = (int **)calloc(height, sizeof(int *));
61     for (i = 0; i < height; i++) {
62         img[i] = (int *)calloc(width, sizeof(int));
63         bufArr[i] = (int *)calloc(width, sizeof(int));
64     }
65     if (img == NULL || bufArr == NULL) {
66         printf("failed to get enough of memory\n");

```

```

67         return -1;
68     }
69
70     for (y = 0; y < height; y++) {
71         for (x = 0; x < width; x++) {
72             fscanf(fpi, "%d", &img[y][x]);
73             bufArr[y][x] = 0;
74         }
75     } // 画像中 (x,y)の位置にある画素をimg[y][x]で参照できる。
76
77     fclose(fpi);
78
79     int x_l, y_u;
80     int x_r, y_d;
81     int value_sum, value_sum_vert, value_sum_hori;
82     int weight;
83     int sobel_vert[3][3] = {{-1, 0, 1}, {-2, 0, 2}, {-1, 0, 1}}; // tate
84     int sobel_hori[3][3] = {{-1, -2, -1}, {0, 0, 0}, {1, 2, 1}}; // yoko
85
86     for (y = 0; y < height; y++) {
87         for (x = 0; x < width; x++) {
88             value_sum = value_sum_vert = value_sum_hori = weight = 0;
89             x_l = y_u = -1;
90             x_r = y_d = 1;
91             if (x == 0) x_l = 0;
92             if (x == width - 1) x_r = 0;
93             if (y == 0) y_u = 0;
94             if (y == height - 1) y_d = 0;
95
96             for (j = y_u; j <= y_d; j++) {
97                 for (i = x_l; i <= x_r; i++) {
98                     weight = sobel_vert[j + 1][i + 1] * DEGREE_OF_EDGE;
99                     value_sum_vert += (weight * img[y + j][x + i]);
100                     weight = sobel_hori[j + 1][i + 1] * DEGREE_OF_EDGE;
101                     value_sum_hori += (weight * img[y + j][x + i]);
102                     if (x == 50 && y == 50) printf("vert : %d, hori : %d, img
: %d\n", value_sum_vert, value_sum_hori, img[y + j][x +
i]);
103                 }
104             }
105             value_sum = (int)sqrt(value_sum_vert * value_sum_vert +
value_sum_hori * value_sum_hori);
106             value_sum = (value_sum > maxvalue) ? maxvalue : value_sum;
107             bufArr[y][x] = (value_sum < 0) ? 0 : value_sum;
108         }
109     }
110
111     for (y = 0; y < height; y++) {
112         for (x = 0; x < width; x++) {
113             img[y][x] = bufArr[y][x];

```

```

114     }
115 }
116
117 // 画像データに対する処理, 各要素の値を+ 30
118 // ただし、画素値が maxvalue より大きい場合、maxvalue にする
119 /*
120 for(y=0; y<height; y++) {
121     for(x=0; x<width; x++) {
122         img[y][x] = img[y][x]+30;
123         if(img[y][x] > maxvalue)
124             img[y][x] = maxvalue;
125     }
126 }
127 */
128 // 画像データをファイル fnameout に出力する
129
130 fpo = fopen(fnameout, "w");
131
132 if (fpo == NULL) printf("Reading file open error ! %s\n", fnameout);
133
134 fprintf(fpo, "P2\n");
135 fprintf(fpo, "# %s\n", fnameout);
136 fprintf(fpo, "%d %d", width, height);
137 fprintf(fpo, " \n");
138 fprintf(fpo, "%d", maxvalue);
139 fprintf(fpo, " \n");
140
141 int count = 0;
142
143 for (y = 0; y < height; y++) {
144     for (x = 0; x < width; x++) {
145         fprintf(fpo, "%3d ", img[y][x]);
146         count++;
147         if (count % 18 == 0) {
148             fprintf(fpo, "\n"); // 1行 18個画素
149         }
150     }
151 }
152 fclose(fpo); // 出力画像をクローズする
153
154 for (i = 0; i < height; i++) {
155     free(img[i]);
156     free(bufArr[i]);
157 }
158 free(img);
159 free(bufArr);
160 }

```

4 実験 2-4

実験 2-4 について周囲長の計算方法を詳しく説明し、各物体の周囲長を示せ。

4.1 考察



図 10: 出力された長さ

図 9: zukei-out.pbm

ソースコード 4: rinkaku-nagasa-pbm.c

```

1 // pbm ファイルから画像データの読込とファイルへの書き出し
2
3 #include <stdio.h>
4 #include <string.h>
5 #include <stdlib.h>
6
7 #define DIRECTION_MAX_INTEGER 8
8
9 int main(void)
10 {
11     FILE *fpi = NULL; // 入力画像
12     FILE *fpo = NULL; // 出力画像
13
14     char buf[258];
15     int width, height; // 画像の横長と縦長
16     int maxvalue; // 画像の最大輝度値
17     int i, x, y;
18     char fnamein[50];
19     char fnamein_buf[50];
20     char fnameout[50];
21     char fnameout_right[10];
22     char *tok;
23     int pro;
24
25     puts("what name is your file? Example name.pbm");
26     scanf("%s", fnamein);
27     for (i = 0; i < sizeof(fnamein_buf); i++) {
28         fnamein_buf[i] = fnamein[i];
29     }

```

```

30 tok = strtok(fnamein_buf, ".");
31 sprintf(fnameout, "%s-out.", tok);
32 tok = strtok(NULL, ".");
33 sprintf(fnameout_right, "%s", tok);
34 strcat(fnameout, fnameout_right);
35
36 // ファイル fnamein から画像データを読み込む
37
38 fpi = fopen(fnamein, "r");
39
40 if (fpi == NULL) printf("Reading file open error ! %s\n", fnamein);
41
42 do {
43     fgets(buf, 256, fpi);
44 } while (buf[0] == '#'); // skip over comments
45
46 if (buf[0] != 'P' || buf[1] != '1') {
47     fclose(fpi);
48     printf(" \n The input image is not pbm format! \n\n");
49     return -1;
50 }
51
52 do {
53     fgets(buf, 256, fpi);
54 } while (buf[0] == '#'); // skip over comments
55
56 sscanf(buf, "%d %d", &width, &height); // 画像サイズを読み込む
57
58 int **img;
59 int **bufArr;
60 img = (int **)calloc(height, sizeof(int *));
61 bufArr = (int **)calloc(height, sizeof(int *));
62 for (i = 0; i < height; i++) {
63     img[i] = (int *)calloc(width, sizeof(int));
64     bufArr[i] = (int *)calloc(width, sizeof(int));
65 }
66 if (img == NULL || bufArr == NULL) {
67     printf("failed to get enough of memory\n");
68     return -1;
69 }
70
71 for (y = 0; y < height; y++) {
72     for (x = 0; x < width; x++) {
73         fscanf(fpi, "%d", &img[y][x]);
74         bufArr[y][x] = 0;
75     }
76 } // 画像中 (x,y)の位置にある画素をimg[y][x]で参照できる。
77
78 fclose(fpi);
79

```

```

80  int dir_current = 6;
81  int dir_next = 4;
82  int skip_frag;
83  int x_c;
84  int y_c;
85  int p_c = 0; // 色 1 黒 0 白
86  int length = 0;
87  for (y = 0; y < height; y++) {
88      for (x = 0; x < width; x++) {
89          if (img[y][x] && !bufArr[y][x] && !p_c) {
90              bufArr[y][x] = 1;
91              x_c = x;
92              y_c = y;
93              dir_current = 6;
94              length = 0;
95              do {
96                  skip_frag = 0;
97                  dir_next = (dir_current - 2 + DIRECTION_MAX_INTEGER) %
                              DIRECTION_MAX_INTEGER;
98                  for (i = 0; i < DIRECTION_MAX_INTEGER; i++) {
99                      if (i)
100                          dir_next = (dir_next + 1 + DIRECTION_MAX_INTEGER) %
                                      DIRECTION_MAX_INTEGER;
101                      switch (dir_next) {
102                          case 0:
103                              if (x_c < width - 1 && img[y_c + 0][x_c + 1]) {
104                                  x_c++;
105                                  skip_frag++;
106                              }
107                              break;
108                          case 1:
109                              if (x_c < width - 1 && y_c > 0 && img[y_c - 1][x_c + 1]) {
110                                  x_c++;
111                                  y_c--;
112                                  skip_frag++;
113                              }
114                              break;
115                          case 2:
116                              if (y_c > 0 && img[y_c - 1][x_c + 0]) {
117                                  y_c--;
118                                  skip_frag++;
119                              }
120                              break;
121                          case 3:
122                              if (x_c > 0 && y_c > 0 && img[y_c - 1][x_c - 1]) {
123                                  x_c--;
124                                  y_c--;
125                                  skip_frag++;
126                              }
127                              break;

```

```

128         case 4:
129             if (x_c > 0 && img[y_c + 0][x_c - 1]) {
130                 x_c--;
131                 skip_frag++;
132             }
133             break;
134         case 5:
135             if (x_c > 0 && y_c < height - 1 && img[y_c + 1][x_c - 1]) {
136                 x_c--;
137                 y_c++;
138                 skip_frag++;
139             }
140             break;
141         case 6:
142             if (y_c < height - 1 && img[y_c + 1][x_c + 0]) {
143                 y_c++;
144                 skip_frag++;
145             }
146             break;
147         case 7:
148             if (x_c < width - 1 && y_c < height - 1 && img[y_c + 1][x_c +
149                 1]) {
150                 x_c++;
151                 y_c++;
152                 skip_frag++;
153             }
154             break;
155         if (skip_frag) {
156             bufArr[y_c][x_c] = 1;
157             dir_current = dir_next;
158             length++;
159             break;
160         }
161     }
162     } while (x_c != x || y_c != y);
163     printf("%d\n", length);
164 }
165 p_c = img[y][x] ? 1 : 0;
166 }
167 }
168
169 for (y = 0; y < height; y++) {
170     for (x = 0; x < width; x++) {
171         img[y][x] = bufArr[y][x];
172     }
173 }
174
175 // 画像データに対する処理, 0,1 逆転
176 /*

```

```

177     for (y = 0; y < height; y++)
178     {
179         for (x = 0; x < width; x++)
180         {
181             if (img[y][x] == 1)
182                 img[y][x] = 0;
183             else
184                 img[y][x] = 1;
185         }
186     }
187     */
188     // 画像データをファイル fnameout に出力する
189
190     fpo = fopen(fnameout, "w");
191
192     if (fpo == NULL) printf("Reading file open error ! %s\n", fnameout);
193
194     fprintf(fpo, "P1\n");
195     fprintf(fpo, "# %s\n", fnameout);
196     fprintf(fpo, "%d %d \n", width, height);
197
198     int count = 0;
199
200     for (y = 0; y < height; y++) {
201         for (x = 0; x < width; x++) {
202             fprintf(fpo, "%1d ", img[y][x]);
203             count++;
204             if (count % 18 == 0) {
205                 fprintf(fpo, "\n"); // 1行 18個画素
206             }
207         }
208     }
209     fclose(fpo); // 出力画像をクローズする
210
211     for (i = 0; i < height; i++) {
212         free(img[i]);
213         free(bufArr[i]);
214     }
215     free(img);
216     free(bufArr);
217 }

```
